

Variational Quantum Regression Math

Graham Burke

June 2025

The main idea of running this program is calculating the inner products of two vectors $\langle x|y\rangle$. We derive that we must use three qubits for the program by understanding our vector has length $N = 8$ and $nqubits = \log_2(N)$ so $nqubits = 3$ in order to encode x or y .

We then calculate the norm of both x and y and normalize in order to convert the states to unit vectors

$$xnorm = ||x|| = (0^2 + 1^2 + 2^2 + \dots 7^2) = ||y|| = ynorm \quad (1)$$

We then divide our x and y vectors by the new calculated $xnorm$ and $ynorm$ in order to calculate the unit vector with value 1.

$$x = \frac{x}{||x||}, y = \frac{y}{||y||} \quad (2)$$

Now that we have established that we begin constructing our quantum circuit. We create said circuit with $nqubits + 1$, where $nqubits$ was calculated above using N and the $+1$ accounts for our ancilla or "control" qubit. Qubits 0-2 encode both x and y and qubit 3 is the ancilla.

The next step is to concatenate the two vectors and divide it by the square root of two.

$$v = \frac{1}{\sqrt{2}} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3)$$

This produces a 16 dimension unit vector, 8 from x and 8 from y . Once concatenated we produce a state:

$$|\phi\rangle = \frac{1}{\sqrt{2}} (|0\rangle \otimes |x\rangle + |1\rangle \otimes |y\rangle) \quad (4)$$

Here $|0\rangle$ and $|1\rangle$ are the ancilla qubit states, $|x\rangle$ and $|y\rangle$ are the remaining encoded qubits.

Once the whole system is initialized we have 4 qubits which produce 16 amplitudes for the superposition. Once the system is initialized we apply a Hadamard gate to the last qubit

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (5)$$

Now we have a combined state:

$$|\tilde{\phi}\rangle = \frac{1}{\sqrt{2}} [|0\rangle \otimes (|x\rangle + |y\rangle) + |1\rangle \otimes (|x\rangle - |y\rangle)] \quad (6)$$

Now once we measure the ancilla qubit we can see the respective states with probability:

$$P(0) = \frac{1}{2} (1 + |\langle x|y\rangle|^2), \quad P(1) = \frac{1}{2} (1 - |\langle x|y\rangle|^2) \quad (7)$$

This sets us up to calculate the inner product later in the program.

In the third block of code we follow the same steps until we get to extracting the state vector. `state.data` is the vector of complex amplitudes from our basis ($2^4 = 16$). Since our `x` and `y` vectors are real we only extract the real parts as seen in the code.

Now we sum the real amplitudes squared for the first half (1)

$$P(\text{ancilla} = 0) = \frac{1}{4} \|x + y\|^2 \quad (8)$$

Where

$$\|x + y\|^2 = \|x\|^2 + \|y\|^2 + 2\langle x|y\rangle = 1 + 1 + 2\langle x|y\rangle \quad (9)$$

So

$$P(0) = \frac{1}{4} (2 + 2\langle x|y\rangle) = \frac{1}{2} (1 + \langle x|y\rangle) \quad (10)$$

Now we return:

$$\langle x|y\rangle = 2P(0) - 1 = 2 \cdot m_{\text{sum}} - 1 \quad (11)$$

Since `x` and `y` are equal their inner product will equal 1.

So:

$$m_{\text{sum}} = \frac{1}{2} (1 + 1) = 1 \Rightarrow \text{Returns } 2 \cdot 1 - 1 = 1 \quad (12)$$

Optimizers

We use five optimizers to minimize our cost function:

$$\text{Cost}(a, b) = \left(1 - \frac{\|y\|}{\|ax + b\|} \cdot \langle y|\text{ansatz}\rangle \right)^2 \quad (13)$$

1: First is the BFGS method (Broyden - Fletcher - Goldfarb - Shanno) which is a gradient based optimizer. It uses a Hessian (a second derivative matrix) and gradients to estimate how much curves can change between each set.

2: COBYLA (Constrained Optimization by Linear Approximations) uses function evaluations instead of derivatives to minimize. This is very useful when gradients are unavailable or when they are unreliable (i.e. noisy quantum systems). Works well when parameters are added.

3: Nelder–Mead compares values instead of using gradients to minimize. It uses a three point triangle to evaluate function values which is great for noisy systems.

4: CG (Conjugate Gradient) is useful in minimizing quadratic functions. It is useful because it can avoid "zigzagging" by keeping directions conjugate or orthogonal in transformed spaces. Works best when the cost function behaves smoothly and quadratically. If so, CG will converge quickly. However, if the cost function is behaving irregularly these gradients and CG may be ineffective.

5: trust-constr (Trust Region Constrained) use non linear optimization by way of trust region methods. It solves smooth nonlinear optimization problems with optional bounds and constraints. It should be noted that it can be used with or without constraints.

Now we want to adapt our ansatz to higher order polynomials:

$$\text{ansatz}(x_i) = a_0 + a_1x_i + a_2x_i^2 + \dots + a_nx_i^n \quad (14)$$

We then have a loop that constructs the whole polynomial of the new ansatz. After the loop we normalize the ansatz and then normalize the ansatz vector and multiply it by the inner product of y and the ansatz:

$$\langle y | \text{ansatz} \rangle = \|y\| \cdot \|\text{ansatz}\| \cdot \langle \hat{y} | \hat{\text{ansatz}} \rangle \quad (15)$$

So this rescales the inner product to reflect the original magnitudes of y and the ansatz (since they were normalized earlier). The return then defines our cost function:

$$\text{Cost} = (1 - \langle y | \text{ansatz} \rangle)^2 \quad (16)$$

This tells us if y and ansatz are equal then the cost will equal 0. If they are orthogonal (i.e. their inner product equals 0) then the cost function will equal 1. If they are opposite the cost function could be high as 4. This encourages high resonance between y and the ansatz.

Now for our final block we are creating a data set where x and y are not equal. Y is a cubic function with added noise.

Set up a random initial guess with order 3 + 1

$$x_0 = [a_0(0), a_1(0), a_2(0), a_3(0)] \quad (17)$$

Now each optimizer tries to minimize using their different methods. Once completed we classically fit the third order polynomial with the unnormalized y. Then we reverse the coefficient order:

$$[a_0, a_1, a_2, a_3] \quad (18)$$

to

$$[a_3, a_2, a_1, a_0] \quad (19)$$

This is for compatibility. Next we find 100 data points between 0 and 7 to help us predict y-values for all fitted models at xfit. We evaluate the classical fit and all optimizers over 100 data points and return the values. We then plot all the points using matplotlib.

VQR in summary predicts output values from input data by using quantum circuits and classical optimizers and is especially useful on noisy systems like quantum computers. We prepare a data set and encode it into a quantum state. Then apply a quantum circuit and use an ansatz. Then we measure the output and calculate the cost function. Then we update our parameters using classical optimizers. We repeat this until we find our desired output.